

Road to Ruby for A Linguistics Nerd

RubyKaigi 2025

作成者: 石田 隼人

英語版は [こちら](#)

最終更新日: 2025年05月15日

自己紹介

- 各種アカウント
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- 職業: ソフトウェアエンジニア
- 趣味
 - 言語学
 - カラオケ
 - 音楽鑑賞
 - 映画鑑賞
 - 猫飼育



免許 / 資格

- 英語
 - TOEIC® Listening & Reading 915点: 2019年12月 取得
- エンジニアリング
 - 情報セキュリティマネジメント: 2017年11月 取得
 - 応用情報技術者: 2017年06月 取得
 - 基本情報技術者: 2016年11月 取得
 - IT パスポート: 2016年04月 取得
- その他
 - 珠算2級: 2002年06月 取得
 - 暗算3級: 2001年02月 取得

RubyKaigi 2025

スキルスタック

- 自然言語
 - 日本語: 母語
 - 英語: ビジネスレベル
- 開発
 - **バックエンド: Ruby(Ruby on Rails), Python**
 - フロントエンド: TypeScript + React.js, TypeScript + Vue.js
 - **データベース: MySQL, PostgreSQL, MongoDB**
 - **アーキテクチャー: モノリス, モジュールモノリス**
 - ホスティング: AWS ESK
 - バージョン管理: Git, GitHub
 - **CI/CD: GitHub Actions, ArgoCD**
 - 監視: Datadog, Sentry

職歴

期間	業種	職種	業務内容
2021年08月- Present	EdTech SaaS	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • CI/CD 運用・保守 • 週次リリースマネージャー • トラブル対応 • ドキュメンテーション • 技術負債解消 • 技術ブログ記事執筆 • LT 登壇
2020年02月- 2020年12月	チャットボット系 SaaS	バックエンド エンジニア	• バックエンド開発 • ドキュメンテーション • 技術負債解消 • 代替 API 検証

職歴

期間	業種	職種	業務内容
2018年06月 - 2020年01月	受託開発	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • トラブル対応 • QA • 技術ブログ記事執筆
2016年04月 - 2018年01月	SES	システムエンジニア	• 全社アカウント管理者 • Windows Server 運用・保守 • セキュリティ管理者助手 • 翻訳・通訳

国際交流活動

- 大学時代
 - 英語学ゼミ(マスメディア英語)
 - 国際交流サークル兼部(2年次)
 - [内閣府主催国際交流プログラム](#)(2013年 - 2016年)
 - 日本語学の授業の S 単位取得(最終年次)
- 海外生活
 - オーストラリアでのワーキングホリデー(2014年04月 - 2015年03月)
 - 2ヶ月間のシドニーの語学学校通学
 - 6ヶ月間の [Hamilton Island Resort](#) での就労
 - 1ヶ月間の NSW 州の [St Ives High School](#) での日本語教師アシスタントボランティア活動
- その他活動
 - 英語での日々の日記(2014年04月 - 現在)
 - [Sunrise Toastmasters Club](#) 参加(2017年02月 - 2018年03月)
 - [Vital Japan](#) 参加(2018年01月 - 2019年07月、2022年10月 - 2023年02月)
 - 英語自己学習
 - Ruby 関連カンファレンス参加

LTに入る前に...

RubyKaigi チーフオーガナイザーの松田明さんより許可を頂き、RubyKaigi 2025 の公式ロゴを背景に使わせて頂いております(著作権侵害はありません)。

Ref) https://x.com/a_matsuda/status/1918612709169672210



コンテンツ

1. [RubyKaigi 2025 の個人的目標](#)
2. [KaigiEffect -沼り分野-](#)
3. [Matz さんとの最も思い出深い議論](#)
4. [As Is vs. To Be](#)
5. [参考文献](#)

RubyKaigi 2025



1. RubyKaigi 2025 の個人的目標

RubyKaigi 2025

1. RubyKaigi 2025 の個人的目標

いち言語学オタクとして Ruby に
コミットするための突破口を見つけること
RubyKaigi 2025

1. RubyKaigi 2025 の個人的目標

この目標に至った経緯

- 長年の Ruby での開発経験はあったが、複雑で難解な低レイヤーの理解から逃げ回ってきた
- この事実により、Ruby を使えはするが最大限活用出来ない未熟な開発者になり果ててしまった
- 私自身、Ruby の低レイヤーのエコシステムやライフサイクルに真剣に向き合い、理解する時が来たことは感じていた
- 何から始めるべきかは分かりかねていたので、RubyKaigi 2025 で言語学的な自分の興味を大きく引く何かを見つけることにした

1. RubyKaigi 2025 の個人的目標

取った戦略

- 興味のある分野だけに絞って観るスピーチを決めた
- 全てを理解しようとはせず、その時に理解出来なかった and/or 興味を引かれたキーワードのみを書き残した
- RubyKaigi 2025 が終了したら、書き残したキーワードの詳細について学び始めた

RubyKaigi 2025



2. KaigiEffect -沼り分野-

RubyKaigi 2025

2. KaigiEffect -沼り分野-

言語処理のエコシステムとライフサイクルへの大きな興味

- 特別な理由はなかったが、かつては興味を持っていなかった
- [@ydah](#) さんの [Dissecting and Reconstructing Ruby Syntactic Structures](#) を拝見し、Ruby のソースコードがどのように処理されるかについて大きな興味を持った

[< Back](#)



Yudai Takada
@ydah_
Ruby committer. A committer of [Lra](#)
[ma](#) LALR parser generator. Co-
Founder of [Kyobashi.rb](#).

JA

Dissecting and Reconstructing Ruby Syntactic Structures

The [parse.y](#) file, a blueprint for creating Ruby's parser, can be considered a direct representation of Ruby's grammatical structure. However, its syntax rules are highly complex, making it challenging to comprehend its architecture.

By dissecting and reconstructing the structure of the grammar rules, we can enhance the maintainability of `parse.y` file. Additionally, through this process, we will gain a deeper understanding of the underlying principles and patterns of Ruby's syntactic structure.

Let's embark on a journey to explore the internal workings of the Ruby language and uncover the true laws and principles that govern its grammar structure.

2. KaigiEffect -沼り分野-

同僚 Rubyists との議論と学習教材の収集

- 言語処理のエコシステムやライフサイクルは私が知っている以上に複雑だった
- 尚のこと、Ruby の言語処理系の詳細について知りたくなった
- それを知るために有用な教材は何かを言語処理に深く精通した同僚の Rubyists にヒアリングと議論をした
- おすすめされたウェブサイトやファイルなどの教材をブックマークしまくった
- X 上でもおすすめをして頂いた(親切な Rubyists に感謝します)

※ 知見に富んだ同僚がいること自体が福利厚生だと実感以上の確信をした

2. KaigiEffect -沼り分野-

同僚 Rubyists との議論と学習教材の収集

Thread

hayat01sh1da 24 Apr at 21:25
yдахさんのRuby Parserのセッションの後に@moppさんに「低レイヤ理解するならこのリポジトリを教材に学習するといいですよ」と教えていただいたリンクを紛失した...
すみません、お手隙の際にもう一度ご共有いただけますでしょうか？ (GW中にやろうと思っていた矢先でした) (edited)

108 replies

hayat01sh1da 24 Apr at 21:27
<https://github.com/rui314/9cc>でしたっけ？

Repository	Stars	Language
rui314/9cc	1866	C

Added by GitHub

mopp 24 Apr at 21:34
あ、それです
<https://www.sigbus.info/compilerbook>を見ながらやるといいますよ

1

ワイがやったやつはこちら
<https://github.com/mopp/9mm>
もう6年前で草も生えない

Repository	Stars	Language
mopp/9mm	22	C

Added by GitHub

1

hayat01sh1da 24 Apr at 21:36
ありがとうございます、めっちゃ助かります！
moppさんは草生やさなくてももう血が通ってるから笑

1

RubyKaigi系のBookmarks整理しなきゃ

2

mopp 24 Apr at 22:02
コンピュータシステムの理論と実装でやるやつめっちゃ難に言うこれです
https://swkagami.hatenablog.com/entry/comput_02mips

swk's log はてな別館
(講義動画) MIPSアーキテクチャ・swk's log はてな別館
前回に引き続き講義動画の公開です。MIPS命令セットを題材として、アセンブリ言語レベルでのコンピュータの動作を学びます。
7 May 2020 (44 kB)

Hatena Blog

コンピュータアーキテクチャ
(講義動画) MIPSアーキテクチャ
swk's log はてな別館

2020-05-07 だからコロナ禍でYouTube動画化されたんだな？

hayat01sh1da 24 Apr at 22:04
ちなみになんですが、「低レイヤを知りたい人のためのCコンパイラ作成入門」を参考書、mmtk-rubyを問題集に手を動かすのと、明快入門 コンパイラ・インタプリタ開発『C処理系を作りながら学ぶ 林端比古実用マスターシリーズ』のとどちらが学習コストがいいですかね？

https://quipper.slack.com/archives/C04NT6AAGJX/p1745499787697829?thread_ts=1745497534.637939&cid=C04NT6AAGJX
無料公開しちゃうのすごい太っ腹...

mopp
2020-05-07 だからコロナ禍でYouTube動画化されたんだな？
From a thread in rubykaigi-ja 24 Apr View reply

mopp 24 Apr at 22:05
mmtkのRuby binding公式であったのか
一番学びたいのはどの部分でしょうか？
mmtkはメモリ管理の分野のうちのGC特化の部分なので、言語処理系の話からはだいぶ離れますね

hayat01sh1da 24 Apr at 22:06
一番学びたいのは構文解析ですね、優先度Sです！
その周辺知識も必要であれば学びたいという感じです。
GCのエコシステムやライフサイクルに明るくないのめかなりコンプレックスなので学びたいですが、優先度B-Aくらいです。 (edited)
Peterさんのプレゼンも理解できるようになりたいです、さっさと行けるけど絶対難しいことやってますよね (edited)

2. KaigiEffect -沼り分野-

同僚 Rubyists との議論と学習教材の収集

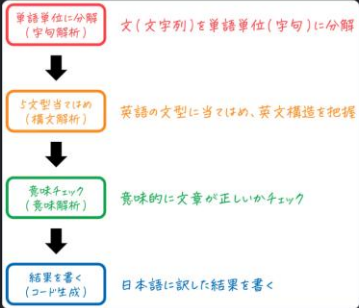
mopp 24 Apr at 22:09

今適当に探したんですが、まずこのあたりの概要を理解するとよさそうです
<https://www.momoyama-usagi.com/entry/info-calc-sys11>

■ 工業大学生ももやまのうさぎ塾

うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ（字句解析と意味解析のしくみ）

こんにちは、ももやまです。今回はコンパイラの処理の流れについて、字句解析と意味解析を中心にまとめていきたいと思います。前提としてオートマトンや文脈自由文法の知識があると理解が早まるかと思います。(136 kB)



1

広告が多いけど普通にわかりやすいな

hayat01sh1da 24 Apr at 22:11

導入が私にとってめっちゃ親近感あります！
オートマトンよく聞かなくて思ってたらかこの周辺の用語だったんですね
自然言語処理独学した時に目にしたのか

mopp 24 Apr at 22:12

あー、どうなのでしょう
言語処理系に应用されることが多いですが
オートマトンは「計算とかなにか？」の表現技法の一つが始まりだった気がします

1

mopp 24 Apr at 22:20

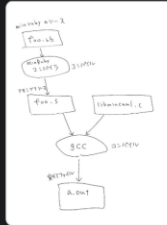
ああ、RubyでつくるRubyがありましたね
構文解析を学びたいなら RubyでつくるRuby だけで十分読めるかもしれません
アセンブラからは逃げられない様子です
<https://blog.agile.esm.co.jp/entry/minruby-compiler>

ESM アジャイル事業部 開発者ブログ

Rubyでつくる、ミニでRubyなコンパイラ - ESM アジャイル事業部 開発者ブログ

こんパイラ〜（挨拶）、電子の海に漂うはかなき泡沫（うたかた）、はたけやまで。みなさん、書籍「RubyでつくるRuby」をご存知ですか？Rubyを使ってRubyのサブセット「MinRuby」のインタプリタを作ることと言語処理系作成のエッセンスを学ぼう！という本です。RubyでつくるRuby
<https://www.lambdanote.com/products/ruby-ruby> 今回は、この本のMinRubyを題材に、簡易なMinRubyコンパイラをRubyで作成してみようと思います。（この記事はESM Advent Calendar 2023 の4日目の記事になります）Gitリボ...

4 Dec 2023



hayat01sh1da 24 Apr at 22:23

おお、そんなに強力な書籍だったとは(さすが遠藤さん)
アセンブラからは逃げられない様子です
もう諦めました、立ち向かいますw

mopp 24 Apr at 22:23

これやるなら3はいらないかもですね
目次を見る限りだと9ccのがやりごたえはあるかもです

1

C言語はねえ
Rubyを書いた後の身には文字列処理がアホほどめんどくさいですよな

2. KaigiEffect -沼り分野-

同僚 Rubyists との議論と学習教材の収集

The screenshot shows a Discord chat interface with a dark theme. At the top, there's a search bar with the text "Tip: Try # F to search this channel". The chat history shows several messages from a user named "ujihisa". The first message is about understanding low-level Ruby. The second message is about Ruby Parser sessions. The third message is about programming language parsers and semantics. The fourth message is about LLVM IR and Ruby VM's iseq. The fifth message is about the importance of understanding Ruby VM's iseq. The sixth message is about the importance of understanding Ruby VM's iseq. The seventh message is about the importance of understanding Ruby VM's iseq. The eighth message is about the importance of understanding Ruby VM's iseq. The ninth message is about the importance of understanding Ruby VM's iseq. The tenth message is about the importance of understanding Ruby VM's iseq. The eleventh message is about the importance of understanding Ruby VM's iseq. The twelfth message is about the importance of understanding Ruby VM's iseq. The thirteenth message is about the importance of understanding Ruby VM's iseq. The fourteenth message is about the importance of understanding Ruby VM's iseq. The fifteenth message is about the importance of understanding Ruby VM's iseq. The sixteenth message is about the importance of understanding Ruby VM's iseq. The seventeenth message is about the importance of understanding Ruby VM's iseq. The eighteenth message is about the importance of understanding Ruby VM's iseq. The nineteenth message is about the importance of understanding Ruby VM's iseq. The twentieth message is about the importance of understanding Ruby VM's iseq. The twenty-first message is about the importance of understanding Ruby VM's iseq. The twenty-second message is about the importance of understanding Ruby VM's iseq. The twenty-third message is about the importance of understanding Ruby VM's iseq. The twenty-fourth message is about the importance of understanding Ruby VM's iseq. The twenty-fifth message is about the importance of understanding Ruby VM's iseq. The twenty-sixth message is about the importance of understanding Ruby VM's iseq. The twenty-seventh message is about the importance of understanding Ruby VM's iseq. The twenty-eighth message is about the importance of understanding Ruby VM's iseq. The twenty-ninth message is about the importance of understanding Ruby VM's iseq. The thirtieth message is about the importance of understanding Ruby VM's iseq. The thirty-first message is about the importance of understanding Ruby VM's iseq. The thirty-second message is about the importance of understanding Ruby VM's iseq. The thirty-third message is about the importance of understanding Ruby VM's iseq. The thirty-fourth message is about the importance of understanding Ruby VM's iseq. The thirty-fifth message is about the importance of understanding Ruby VM's iseq. The thirty-sixth message is about the importance of understanding Ruby VM's iseq. The thirty-seventh message is about the importance of understanding Ruby VM's iseq. The thirty-eighth message is about the importance of understanding Ruby VM's iseq. The thirty-ninth message is about the importance of understanding Ruby VM's iseq. The fortieth message is about the importance of understanding Ruby VM's iseq. The forty-first message is about the importance of understanding Ruby VM's iseq. The forty-second message is about the importance of understanding Ruby VM's iseq. The forty-third message is about the importance of understanding Ruby VM's iseq. The forty-fourth message is about the importance of understanding Ruby VM's iseq. The forty-fifth message is about the importance of understanding Ruby VM's iseq. The forty-sixth message is about the importance of understanding Ruby VM's iseq. The forty-seventh message is about the importance of understanding Ruby VM's iseq. The forty-eighth message is about the importance of understanding Ruby VM's iseq. The forty-ninth message is about the importance of understanding Ruby VM's iseq. The fiftieth message is about the importance of understanding Ruby VM's iseq. The fifty-first message is about the importance of understanding Ruby VM's iseq. The fifty-second message is about the importance of understanding Ruby VM's iseq. The fifty-third message is about the importance of understanding Ruby VM's iseq. The fifty-fourth message is about the importance of understanding Ruby VM's iseq. The fifty-fifth message is about the importance of understanding Ruby VM's iseq. The fifty-sixth message is about the importance of understanding Ruby VM's iseq. The fifty-seventh message is about the importance of understanding Ruby VM's iseq. The fifty-eighth message is about the importance of understanding Ruby VM's iseq. The fifty-ninth message is about the importance of understanding Ruby VM's iseq. The sixtieth message is about the importance of understanding Ruby VM's iseq. The sixty-first message is about the importance of understanding Ruby VM's iseq. The sixty-second message is about the importance of understanding Ruby VM's iseq. The sixty-third message is about the importance of understanding Ruby VM's iseq. The sixty-fourth message is about the importance of understanding Ruby VM's iseq. The sixty-fifth message is about the importance of understanding Ruby VM's iseq. The sixty-sixth message is about the importance of understanding Ruby VM's iseq. The sixty-seventh message is about the importance of understanding Ruby VM's iseq. The sixty-eighth message is about the importance of understanding Ruby VM's iseq. The sixty-ninth message is about the importance of understanding Ruby VM's iseq. The seventieth message is about the importance of understanding Ruby VM's iseq. The seventy-first message is about the importance of understanding Ruby VM's iseq. The seventy-second message is about the importance of understanding Ruby VM's iseq. The seventy-third message is about the importance of understanding Ruby VM's iseq. The seventy-fourth message is about the importance of understanding Ruby VM's iseq. The seventy-fifth message is about the importance of understanding Ruby VM's iseq. The seventy-sixth message is about the importance of understanding Ruby VM's iseq. The seventy-seventh message is about the importance of understanding Ruby VM's iseq. The seventy-eighth message is about the importance of understanding Ruby VM's iseq. The seventy-ninth message is about the importance of understanding Ruby VM's iseq. The eightieth message is about the importance of understanding Ruby VM's iseq. The eighty-first message is about the importance of understanding Ruby VM's iseq. The eighty-second message is about the importance of understanding Ruby VM's iseq. The eighty-third message is about the importance of understanding Ruby VM's iseq. The eighty-fourth message is about the importance of understanding Ruby VM's iseq. The eighty-fifth message is about the importance of understanding Ruby VM's iseq. The eighty-sixth message is about the importance of understanding Ruby VM's iseq. The eighty-seventh message is about the importance of understanding Ruby VM's iseq. The eighty-eighth message is about the importance of understanding Ruby VM's iseq. The eighty-ninth message is about the importance of understanding Ruby VM's iseq. The ninetieth message is about the importance of understanding Ruby VM's iseq. The ninety-first message is about the importance of understanding Ruby VM's iseq. The ninety-second message is about the importance of understanding Ruby VM's iseq. The ninety-third message is about the importance of understanding Ruby VM's iseq. The ninety-fourth message is about the importance of understanding Ruby VM's iseq. The ninety-fifth message is about the importance of understanding Ruby VM's iseq. The ninety-sixth message is about the importance of understanding Ruby VM's iseq. The ninety-seventh message is about the importance of understanding Ruby VM's iseq. The ninety-eighth message is about the importance of understanding Ruby VM's iseq. The ninety-ninth message is about the importance of understanding Ruby VM's iseq. The hundredth message is about the importance of understanding Ruby VM's iseq.

ujihisa 22:57
低レイヤ理解と聞いて
RubyでつくるRubyとかでは始めるのがスムーズかもです
1

ujihisa 22:59
(前提として発端がRuby Parserのセッションなのでそちらがわに興味があると推測しています)
1 reply 9 days ago

ujihisa 23:00
プログラミング言語のparserならびにそれらのセマンティクスについていろいろあれする系は僕がもっとも好きな分野のうちの一つです 🙄
1

あと、Cないしアセンブリについて勉強するときに、あえてLLVM IRとかも軽く見ておくと勉強になるかもです
(SSAとか各地でいろいろ言及されるので)
Ruby VMのiseqは重要なものですが、いきなりそこをやるよりも比較対象になる他のものを知っておくと便利かな的な

hayat01sh1da 23:02
本家すごいボリュームですね！
<https://llvm.org/docs/LangRef.html>

ujihisa 23:02
そうです、全部理解するのは無理とあきらめて、概要と簡単な使い方だけ把握すれば十分だと思います！
たとえばですが、超簡単な独自言語を考えて、
それをアセンブリ、iseq、llvm ir、などそれぞれに変換するプログラムを作ってみるとか
ここでいう超簡単というのは、ループも制御構文もなく、変数と四則演算とstdout出力しかないみたいなイメージです (edited)
1

こういった経験をする、ソースコードをみたときに4つぐらいのレイヤでそれぞれ別のものが見えるようになると思います

hayat01sh1da 23:07
ソースコードをみたときの世界観がまるで変わるんですね、面白い！

2. KaigiEffect -沼り分野-

同僚 Rubyists との議論と学習教材の収集

ujihisa 23:07

- A. プログラムの持つ「意味」をみたレイヤ (人間の脳が認識している世界)
- B. ソースコードそのまま全体を文字列としてみたレイヤ (人間の目が認識している世界)
- C. トークン列として見たレイヤ (parserが見ている世界、またはテキストエディタのシンタックスハイライト機構などが見ている世界)
- D. ASTとして見たレイヤ (狭義のcompilerが見ている世界)
- E. 内部の命令列として見たレイヤ (runtimeが見ている世界)

(edited)

書いてたら5つになっちゃった

hayat01sh1da 休 23:10

2つめが私(石田その①)、1・3つ目が TomPng さん(石田その②w)ってイメージを持ちました(TomPng さんは全部分かった上でトークン列に関心をおいていると思いますが)

内部の命令列として見たレイヤ (runtimeが見ている世界)

今年の Class#new 高速化でアーロンさんがお話しされてた記憶があります！

1

ujihisa 23:12

たとえばMatzさんが言語のデザインについて考えているときはAとBで、その互換性について考えるときや実装するときは主にCとDで (edited)

1

Aaronさんがobject生成まわりについて考えている (必然的にGCまわりについても考えている)ときは実質Eのみ、みたいな感じですね!

なるほど 1

2. KaigiEffect -沼り分野-

知識インプット教材

- [LLVM Language Reference Manual](#)
- [Parsing Techniques](#)
- [低レイヤを知りたい人のための C コンパイラ作成入門](#)
- [MIPS アーキテクチャ](#)
- ✓ [うさぎでもわかる計算機システム Part10 コンパイラの基礎・分割コンパイルの流れ](#)
- ✓ [うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ\(字句解析と意味解析のしくみ\)](#)
- ✓ [Ruby Parser 開発日誌\(14\) - LR parser 完全に理解した](#)
- ✓ [オートマトンと言語理論 学習記 #1: オートマトンとは何か-](#)
- ✓ [オートマトンと言語理論 学習記 #2: 有限オートマトン-](#)
- ✓ [Ruby の中身の予備知識 RubyKaigi の前に知り鯛！](#)
- ✓ [たのしい Ruby の構文解析ツアー](#)
- ✓ [LL 法と LR 法の違いは？調べてみた！-完全版-/Comparing LL and LR parse algorithm -EX Edition-](#)

2. KaigiEffect -沼り分野-

知識アウトプット教材

- ✓ [Ruby でつくる、ミニで Ruby なコンパイラ](#)
- [9cc](#)
- [lr-parser-101](#)
- [『Ruby でつくる Ruby ゼロから学びなおすプログラミング言語入門』](#)

RubyKaigi 2025

2. KaigiEffect -沼り分野-

新生 Ruby コミュニティ参加

[Shikumi.rb](https://shikumi.rb)

organised by [@hashimoto-kei](https://twitter.com/hashimoto-kei) and [@utsubo](https://twitter.com/utsubo)

- 2025年05月に誕生したばかりのコミュニティ
- セッションは主にオンライン開催
- Ruby の言語処理の仕組みを勉強することを主眼に置いているので、KaigiEffect に当てられた私には渡りに船だった
- 目下皆勤賞(2025年05月09日 現在)

3. Matz さんとの最も思い出深い議論

RubyKaigi 2025

3. Matz さんとの最も思い出深い議論

以前「[言語の生と死 - 自然言語 vs. プログラミング言語 -](#)」で言及したように、Matz さんが Twitter(現: X)にて私のアドベントカレンダーの記事を引用して下さるくらい、言語学に関心をお持ちであることは存じ上げていた。

また、以前「[比較言語学のススメ - Ruby vs. Python に学ぶ言語のアイデンティティ -](#)」で言及したように、私自身も Ruby は自然言語の英語のような滑らかな「書き味」を強く意識していることには確信を持っていた。



3. Matz さんとの最も思い出深い議論

株式会社アンドパッド様主催の[コード懇親会](#)が渡りに船となり、同じ島のテーブルで一緒した Matz さんと以下のような議論を交わすことが出来た。

※ H = hayat01sh1da, M = Matz さん

H) 「松本さんは Perl に倣って Ruby を開発されたと記憶しており、ゆえに Perl といくつかの類似点があると認識しています。その1つが[言語学・認知学的考慮](#)ですが、**Ruby を開発し、育て、世にリリースする際にそのような言語学・認知学的な哲学をお持ちだったのでしょうか？」**

M) 「Perl の開発者である Larry Wall さんは言語学者としてのバックボーンがありますが、一方で私はコンピューターサイエンス専攻でした。バックグラウンドはお互いに違いますが、[言語学・認知学的考慮](#)を互換性に注意を払いながら強く意識しています。」

3. Matz さんとの最も思い出深い議論

H) 「1人の Rubyist・英語学専攻者として、Ruby 自然言語の英語のような滑らかな『書き味』を提供していると考えています。**Ruby を書くことは英語で詩を書くの**にかなり近い体験があると感じます。松本さんの言語学・認知学的哲学を理解しましたし、『[比較言語学のススメ -Ruby vs. Python に学ぶ言語のアイデンティティ-](#)』で述べた仮説が正しかったと証明出来ました。」

M) 「私はメソッドがどのように動くかだけでなく、そのメソッドが何の目的で動作するのも重要視しています。例えば、[Enumerable#collect](#) と [Enumerable#map](#) について考えてみましょう。両者ともにレシーバーの各要素のブロック内での処理結果を新しい配列にして返すメソッドです。前者はデータの集合体を生成する、後者はレシーバーの各要素とブロック内の処理結果をマップするという目的があります。この例から分かるように、同じデータ処理をするのにも様々な目的を孕んでいるという訳です。」

H) 「日本語版の『[プログラマが知るべき97のこと](#) > [107. 名前重要](#)』で松本さんが仰っていた『命名は設計を如実に反映する』という事実を連想しました。」

3. Matz さんとの最も思い出深い議論

M) 「命名の観点で、私は過去に1つ失敗したことがあります。私はその時、レシーバー自信をブロック引数に取り、その処理結果を返却するメソッドの命名を考えていました。初めはそれに `Object#yield self` という名前を付けました。『自身渡す』という挙動の観点では意味を成しています。しかし、それ以前の『それが何をするのか』を命名に反映出来ていませんでした。後に `Object#then` というエイリアスを作りました。データ処理はレシーバー以前で一旦完了しており、文字通り『それから』何をするかに焦点を当てるような命名にしたのです。これによって、メソッドがどう動くかだけでなく、それが何を目的としてるかが分かりやすくなりました。」

H) 「完全理解です。英文構造で言うと多くのメソッドはレシーバーを主語、メソッド自身を述語動詞、引数を目的語、ブロックを従属節(名詞節・形容詞節・副詞節)として扱う理解をしています。しかし、松本さんは副詞である 'then' をメソッドとして、ブロック内を並列する主節として扱う文構造を採用されたのが目から鱗でした！」

3. Matz さんとの最も思い出深い議論

句動詞として機能

```
1  [10.downto(1)].yield self [{ it.sort }]
2  => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4  10.downto(1).then { it.sort }
5  => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
6
```

- S = 主語
- V = 述語動詞
 - V₁: SV
 - V₂: SVC
 - V₃: SVO
 - V₄: SVO₁O₂
 - V₅: SVOC
- O = 目的語
 - O₁: 間接目的語
 - O₂: 直接目的語
- C = 補語
- [従属節]
- (修飾語句)

※ [Enumerable#sort](#) は返り値として新しい配列を [enum_sort](#) 関数を通して [rb_ary_sort_bang](#) 関数で返却するので、Range オブジェクトは Array オブジェクトに暗黙的に変換される。

3. Matz さんとの最も思い出深い議論

Object#yield_self vs. Object#then

- **Object#yield_self**

- 実際にレシーバー自身を後続のブロック引数として渡す
- ブロック内でそれがどのように扱われるかは命名からは分からない
- ブロックは従属名詞節として機能しその中で **SV₁** 構造を持つため、入れ子構造を持つ複文が高い認知負荷をもたらす

- **Object#then**

- 文字通り *and then* として機能する(等位接続詞 + 副詞)
- 1つ目の主節全体を参照する人称代名詞 *it* に対する処理に焦点を当てている
- そのためブロック内で *it* に対する処理が行われることが命名から分かる
- ブロックはもう1つの並列な主節として機能するので、入れ子構造が存在しない単文がより良い可読性がもたらす

3. Matz さんとの最も思い出深い議論

Matz さんとの議論は10分と少し程だったが、実りある時間を持って大変光栄であった。

Matz さんには、大の Ruby 好き・言語学オタクとして感謝に堪えません。





4. As Is vs. To Be

RubyKaigi 2025

4. As Is vs. To Be

AsIs: 現状

- Ruby でウェブアプリケーションやスクリプトを実装するだけの Rubyist

ToBe: 目標

- Ruby の低レイヤーエコシステムやライフサイクルを理解し、ウェブアプリケーションやスクリプトのパフォーマンスを最適化出来る Rubyist
- Ruby の言語処理系がどのように動くのかを理解し改善のためのコミットが出来る Rubyist
- Ruby の言語学・認知学観点での設計を豊かに、そして最適化出来る Rubyist



5. 参考文献

RubyKaigi 2025

5. 参考文献

- [RubyKaigi 2025 > Schedule](#)
 - 最終アクセス日: 2025年05月05日
- [LLVM Language Reference Manual](#)
 - 最終アクセス日: 2025年05月05日
- [PARCING TECHNIQUES -A Practical Guide-](#)
 - 最終アクセス日: 2025年05月05日
- [低レイヤを知りたい人のためのCコンパイラ作成入門](#)
 - 最終アクセス日: 2025年05月05日
- [MIPS アーキテクチャ](#)
 - 最終アクセス日: 2025年05月05日

5. 参考文献

- [うさぎでもわかる計算機システム Part10 コンパイラの基礎・分割コンパイルの流れ](#)
 - 最終アクセス日: 2025年05月05日
- [うさぎでもわかる計算機システム Part11 コンパイラの処理の流れ\(字句解析と意味解析のしくみ\)](#)
 - 最終アクセス日: 2025年05月05日
- [Ruby Parser 開発日誌\(14\) - LR parser 完全に理解した](#)
 - 最終アクセス日: 2025年05月05日
- [オートマトンと言語理論 学習記 #1: オートマトンとは何か -](#)
 - 最終アクセス日: 2025年05月05日
- [オートマトンと言語理論 学習記 #2: 有限オートマトン -](#)
 - 最終アクセス日: 2025年05月05日

5. 参考文献

- [Ruby の中身の予備知識 RubyKaigi の前に知り鯛！](#)
 - 最終アクセス日: 2025年05月05日
- [たのしい Ruby の構文解析ツアー](#)
 - 最終アクセス日: 2025年05月05日
- [LL 法と LR 法の違いは？調べてみた！-完全版-/Comparing LL and LR parse algorithm -EX Edition-](#)
 - 最終アクセス日: 2025年05月05日
- [9cc](#)
 - 最終アクセス日: 2025年05月05日
- [lr-parser-101](#)
 - 最終アクセス日: 2025年05月05日

5. 参考文献

- 遠藤侑介、『[Ruby でつくる Ruby ゼロから学びなおすプログラミング言語入門](#)』、東京、2017年
- [Ruby 3.4 リファレンスマニュアル > 組み込みライブラリ](#)
 - Last Accessed On: 05 May 2025
- [Perl 6 Design Philosophy](#)
 - 最終アクセス日: 2025年05月05日
- [97 Things Every Programmer Should Know](#)
 - 最終アクセス日: 2025年05月05日
- [プログラマが知るべき97のこと](#)
 - 最終アクセス日: 2025年05月05日



EOD

RubyKaigi 2025